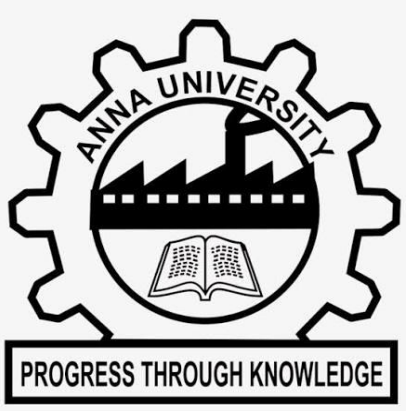
STUDENT FEEDBACK SYSTEM USING PYTHON

A SUMMER INTERNSHIP REPORT

Submitted by

E. KESAVAN (810023205087)



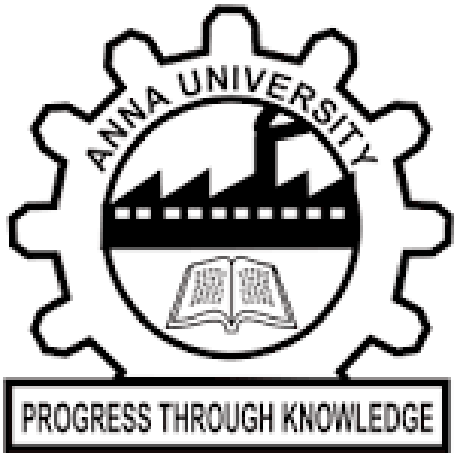
DEPARTMENT OF INFORMATION TECHNOLOGY

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY, TIRUCHIRAPPALLI - 620 024

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS), ANNA UNIVERSITY, TIRUCHIRAPALLI – 620024.

DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **" Student Feedback System using python"** is the Bonafide work of **E.KESAVAN (810023205087)** and has satisfactorily completed the course of **IT3711 - SUMMER INTERNSHIP** for **SEVENTH** semester, **B.Tech. INFORMATION TECHNOLOGY** during the academic year 2026-2027**.**

Faculty in charge **Head of the department**

Submitted for the practical examination held on

ABSTRACT

Student feedback is an important mechanism for understanding the quality of teaching, course delivery, practical sessions, learning materials, and overall academic experience. When feedback is collected manually through paper forms or disconnected spreadsheets, the process can become time-consuming and difficult to analyze. The proposed Student Feedback System using Python provides a structured digital approach for collecting, storing, validating, and reviewing student feedback.

The system is designed around a Python application layer, a simple user interface, and a relational database. Students can authenticate themselves, select a course, answer predefined feedback questions using rating values, and submit comments. The application validates the input before storing the response. Administrators can view summarized feedback and use the collected information to identify areas requiring improvement.

The project demonstrates the practical use of Python programming, GUI development, database operations, input validation, modular programming, and software testing. SQLite is suitable for a lightweight prototype because it provides disk-based database storage without requiring a separate database server. Tkinter is used as a standard Python interface for GUI development.

Keywords: Python, Student Feedback, Tkinter, SQLite, GUI, Database, Rating System, Software Testing

TABLE OF CONTENTS

Chapter	Title	Page
1	Introduction & Objectives	
2	Daily work Report	
3	Literature Review / System Study	
4	System Requirements & Architecture	
5	Development & Result Analysis	
6	Conclusion	
7	Future Work	
8	References	

CHAPTER 1: INTRODUCTION & OBJECTIVES

1.1 Profile of the Organization



FANTASY SOLUTION

Fantasy solution as a leading IT solution and service provider, provides innovative information technology - enabled solutions and services to meet the demands arising from social transformation, shaping new life styles for individuals and creating values for the society.

Focusing on software technology, Fantasy solution provides industry solutions and product engineering solutions, related software products & platforms, and services, through seamless integration of software and services, software and manufacturing, as well as technology and industrial management capacity.

Fantasy solution helps industry customers establish best practices in business development and management. The fantasy solution serves include real time projects, web designing, web hosting, software development and training etc, in many of which, has a leading market share. Notably, Fantasy Solution has participated in the formulation of many national IT standards and specifications.

Fantasy solution has the world's leading product engineering capabilities, ranging from consultation, design, R&D, and integration to testing of embedded software, in the fields of automotive electronics, smart devices, digital home products, and IT products. The software provided by fantasy solution runs in a number of globally renowned brands.

Particularly offering the services that include application development & maintenance, ERP implementation & consulting, testing, performance

engineering, software localization & globalization, IT infrastructures, BPO, IT education & training, etc. Sticking to its business philosophy and brand commitment of "Beyond Technology", fantasy solution provides innovative information technologies to drive the sustainable development of society, as well as becoming a company that is well recognized and respected by employees, shareholders, customers, and society.

.

1.1 Objectives of the Internship

- Understand the software development lifecycle followed in a professional environment.
- Strengthen Python programming and modular application development skills.
- Design and implement a digital student feedback workflow.
- Learn database creation, insertion, retrieval, and update operations.
- Apply input validation and basic security practices.
- Perform functional testing and analyze system results.
- Improve technical documentation and professional communication.

1.2 Introduction

The Student Feedback System is intended to replace a repetitive manual feedback process with a consistent digital workflow. The system captures feedback in a structured format so that responses can be stored and summarized efficiently. Python was selected because it provides readable syntax, a rich standard library, and practical GUI and database interfaces. The Python Software Foundation maintains the Python language and its ecosystem, while the official documentation provides the language tutorial and standard-library references used during development

DAILY WORK REPORT

DAY: 1

I gained knowledge of Python's fundamental data types today, including texts, integers, and Boolean values. I worked on simple print statements and math computations.

DAY: 2

I learned how to assign and reassign values, understand the standards around variable names, and became comfortable working with variables in Python. I worked on manipulating and saving data using variables.

DAY: 3

Today, I learnt how to use control flow statements, for and while loops, and if-else expressions in Python. I attempted to use these statements to manage the execution of my programs.

DAY: 4

I concentrated on learning the basic list operations—slicing, adding, and extending—as well as how to construct, alter, and retrieve items of lists using their indices. I worked on modifying lists in order to store and retrieve data.

DAY: 5

I started experimenting with Python functions today. I may now declare, call, pass, and receive the return values of functions. I invested time in creating reusable functions to finish particular jobs.

DAY: 6

I looked at string concatenation and slicing in Python, as well as the string operations higher(), lower(), and len (). I worked on changing the contents of strings and extracting certain data from them.

DAY: 7

I concentrated on using dictionaries in Python today. I gained knowledge on how to construct dictionaries, use keys to access and edit items, and carry out standard dictionary operations like adding and deleting key-value pairs. To effectively store and retrieve data, I worked using dictionaries.

DAY: 8

When I first began working with sets in Python, I became familiar with their special ability to contain only distinct elements as well as the union, intersection, and difference operations that could be performed on them. I worked on performing typical set operations with sets.

DAY: 9

I learned about file handling in Python today. Along with learning how to solve problems, I also learned how to open, read, write, and close files. Using a range of file handling techniques, I practiced reading data from files and writing output to files.

DAY: 10

My main interest was working with Python modules, i.e., importing, using, and creating custom and built-in modules. I worked on organizing my code with modules and having access to pre-built functionality.

DAY: 11

I started studying Python exception handling today. I gained knowledge of finally blocks, try-except blocks, and managing certain exceptions. In order to manage mistakes in my apps with grace, I practiced handling exceptions.

DAY: 12

I concentrated on learning about encapsulation, inheritance, and polymorphism—three fundamental concepts of object-oriented programming—using classes and objects in Python. My tasks included class creation, attribute and method definition, and object instantiation.

DAY: 13

I learned how to organize and structure code into reusable modules and packages in Python by working with modules and packages today. I grew proficient at importing modules, making use of their features, and building my own packages.

DAY: 14

I experimented with using external libraries in Python and learned how to use pip and other tools to install and use third-party libraries. I worked on improving my Python programs by utilizing web scraping, data analysis, and visualization components.

DAY: 15

Today, I concentrated on using Python's built-in data structures, sets and tuples. I gained knowledge of their traits, benefits, and applications. I created and modified tuples and sets for various tasks.

DAY: 16

My focus completely shifted to using Python's file input/output functions to read and write data to CSV, JSON, and other file formats. I had to import data from files, make edits, and then save the finished products back into files for my task.

DAY: 17

Today, I looked at using two Python modules for data analysis and manipulation: NumPy and Pandas. I learned how to do operations like data sorting, filtering, and aggregation, and I obtained experience with dataset analysis and visualization.

DAY: 18

Although I mostly used Python's loops and conditional expressions, I also learned how to use new control flow statements like nested loops and elif. I worked on developing more intricate and adaptable program logic utilizing these building blocks.

DAY: 19

I gained knowledge today about working with dates and times in Python, including formatting, altering, calculating time intervals, and managing time zones. I worked on time management and date computation.

DAY: 20

I experimented with searching, matching, and modifying text patterns using regular expressions and Python's re package. I worked with regular expressions to change text, retrieve information, and validate input.

DAY: 21

I spent much of today studying how to utilize Python to work with databases. In particular, I learnt how to establish connections, run queries, and retrieve data using programs like sqlite3 and SQL Alchemy. I worked on using databases and carrying out simple CRUD tasks.

DAY: 22

I became interested in Python's Application Programming Interfaces and started studying how to use them for HTTP calls, handling responses, and processing data from online APIs. I invested a lot of time learning how to use third-party APIs in my Python programs.

DAY: 23

I worked with Python online scraping for the majority of the day, learning how to use programs like Beautiful Soup and Scrapy to extract data from webpages. I made choices, worked my way through the HTML structure, and pulled out data for automation or analysis.

DAY: 24

I started using Python to work with data visualization, learning how to make plots, graphs, and charts with tools like Matplotlib and Seaborn. I concentrated on data visualization to help me understand the data and effectively communicate my findings.

DAY: 25

I learned how to manage dependencies and establish isolated environments for various projects using tools like virtualenv or conda in today's Python virtual environment workshop. My attention was devoted to configuring and maintaining virtual environments.

DAY: 26

My main objective was to master the use of generators and decorators in Python, two tools that may be used to expand or change a function's behavior. I took my time incorporating generators and decorators into my code.

DAY: 27

Today, I began learning about async and await keywords, event loops, coroutines, and asynchronous programming in Python. In order to increase responsiveness and performance under specific conditions, I concentrated on writing asynchronous code.

DAY: 28

I thought about using one of the Python web frameworks, Flask or Django. My understanding of routing, question-answering, and producing dynamic online content increased. I worked on creating basic web applications using a chosen web framework.

DAY: 29

Today, I concentrated on using the Python testing frameworks, unit test and pytest. I can now create assertions, write test cases, and automate testing procedures. I took my time reviewing my code to make sure it was reliable and accurate.

DAY: 30

My beginner's Python internship comes to an end today. I took the day to think back on everything I had learned, found, and achieved. I evaluated my progress, identified my areas of weakness, and prepared for the final test or presentation.

CHAPTER 2: LITERATURE REVIEW / SYSTEM STUDY

2.1 Evaluation of Existing Frameworks

Traditional student feedback collection is generally carried out using paper forms, spreadsheets, or generic online form platforms. In paper-based systems, students manually fill out feedback forms, which are later collected and processed by faculty members. Although this method is simple and does not require technical infrastructure, it involves considerable manual effort, requires physical storage, and makes data analysis time-consuming.

Spreadsheet-based systems provide better organization and allow faculty members to perform calculations, sorting, filtering, and basic statistical analysis. However, manual data entry is still required, and problems such as duplicate records, incorrect entries, accidental modification, and version-control issues may occur when multiple users handle the same file.

Generic online form systems provide a faster method for collecting feedback. Responses can be stored electronically and exported for further analysis. However, these systems may not provide features specifically designed for academic feedback, such as customized feedback categories, automatic evaluation, department-wise analysis, subject-wise reports, and controlled access.

The existing approaches are therefore useful for basic feedback collection but have limitations in terms of automation, validation, analysis, and report generation. The proposed **Student Feedback System Using Python** addresses these limitations by providing a customized application for collecting, storing, validating, analyzing, and generating reports from student feedback.

Existing Approach	Advantages	Limitations
Paper Forms	Simple, familiar, works offline	Manual entry, physical storage, slow analysis
Spreadsheets	Easy calculations, sorting, and filtering	Data duplication, version-control issues, manual processing
Generic Online Forms	Quick collection and easy distribution	Limited customization and domain-specific analysis
Proposed Python System	Automated processing, validation, analysis, and reporting	Requires software development and maintenance

2.2 Problem Statement Identification

The study of existing student feedback methods reveals several operational and technical limitations. In traditional paper-based feedback systems, faculty members have to manually collect forms and enter the responses into another system for analysis. This increases the workload and may introduce human errors during data entry.

Spreadsheet-based feedback systems reduce some of the manual work but still require considerable effort for organizing and maintaining the data. Duplicate entries, incorrect values, accidental changes, and inconsistent formatting can affect the accuracy of the results. Generating detailed reports from large amounts of feedback data can also become difficult and time-consuming.

Generic online forms provide convenient data collection, but they may not support all the requirements of an academic feedback system. They may lack

customized validation, automatic statistical analysis, subject-wise evaluation, and structured report generation.

The major problems identified in the existing system are:

1. **Manual data entry and processing** increase staff workload.
2. **Duplicate feedback entries** may affect the accuracy of results.
3. **Invalid or incomplete responses** may be stored without proper validation.
4. **Manual calculation of feedback scores** consumes time.
5. **Report generation** requires additional processing and formatting.
6. **Difficulty in organizing feedback data** based on subjects, faculty, classes, or departments.
7. **Limited automation** in conventional feedback collection methods.
8. **Data management and storage issues** can occur when large amounts of feedback are collected.
9. **Lack of a customized academic workflow** in generic feedback platforms.

Therefore, there is a need for an automated and customized student feedback system that can simplify feedback collection, reduce manual processing, improve data accuracy, and generate meaningful reports efficiently.

2.3 Proposed Solution / Methodology

The proposed **Student Feedback System Using Python** is designed to automate the process of collecting, storing, validating, analyzing, and reporting student feedback. The system can be developed as a Python-based application with a user-friendly interface for students and administrators or faculty members.

The system allows students to enter their feedback through a structured interface. The application validates the submitted information before storing it in a database. Appropriate checks can be implemented to prevent incomplete

submissions and duplicate feedback entries.

The collected feedback is stored in a structured database such as **SQLite**. Python can be used to perform data processing and statistical calculations, including calculating average ratings for different questions, subjects, or faculty members. The system can also generate summarized results that help administrators understand the overall feedback.

The proposed methodology consists of the following stages:

Student Login/Input → Feedback Form → Data Validation → Duplicate Checking → Database Storage → Data Processing → Score Calculation → Report Generation

The system can use **Python and Tkinter** for the graphical user interface and **SQLite** for local database management. Python libraries such as **Pandas** can be used for data processing and **Matplotlib** can be used for graphical representation of feedback results.

Major Features of the Proposed System

- Student feedback submission
- User-friendly feedback interface
- Duplicate feedback detection
- Secure and structured data storage
- Automatic calculation of feedback scores
- Subject-wise and faculty-wise analysis
- Graphical representation of results
- Automated report generation
- Easy retrieval and management of feedback records

The proposed system reduces manual effort and improves the efficiency and accuracy of feedback analysis. By integrating data collection, validation, storage, analysis, and reporting into a single Python-based application, the system provides a more organized and effective approach to student feedback management.

CHAPTER 3: SYSTEM REQUIREMENTS & ARCHITECTURE

3.1 Hardware & Software Prerequisites

The **Student Feedback System Using Python** is designed as a lightweight software application and does not require high-end hardware. The system can be developed and executed on a standard desktop or laptop computer. The following hardware and software requirements are recommended for development, testing, and deployment.

3.1.1 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3 / AMD equivalent	Intel Core i5 or above
RAM	4 GB	8 GB or above
Storage	10 GB free space	20 GB or above
Display	1366 × 768 resolution	Full HD 1920 × 1080
Keyboard & Mouse	Standard	Standard
GPU	Integrated graphics	Integrated graphics sufficient
Network	Not mandatory for local application	Internet connection for development and updates

Since the proposed system mainly performs form processing, database operations, statistical calculations, and report generation, a dedicated GPU is not required.

3.1.2 Software Requirements

Software / Technology	Requirement / Version
Operating System	Windows 10/11, Linux, or macOS

Programming Language	Python 3.10 or above
GUI Framework	Tkinter
Database	SQLite
Data Processing	Pandas
Data Visualization	Matplotlib
IDE / Code Editor	Visual Studio Code / PyCharm
Database Tool	SQLite Browser / DB Browser for SQLite
Testing	Python testing tools / manual functional testing
Version Control	Git (optional)

3.1.3 Technology Description

Python:

Python is used as the primary programming language for developing the application. It provides a simple syntax and supports libraries required for database management, data processing, visualization, and report generation.

Tkinter:

Tkinter is used to create the graphical user interface. It provides components such as labels, buttons, text fields, radio buttons, combo boxes, and tables required for the feedback application.

SQLite:

SQLite is used to store student, subject, faculty, and feedback information. It is lightweight and does not require a separate database server, making it suitable for a desktop-based application.

Pandas:

Pandas can be used for organizing and processing feedback data and performing calculations such as averages and summary statistics.

Matplotlib:

Matplotlib can be used to represent feedback results using bar charts, pie charts, and other graphical representations.

3.2 Architectural Design

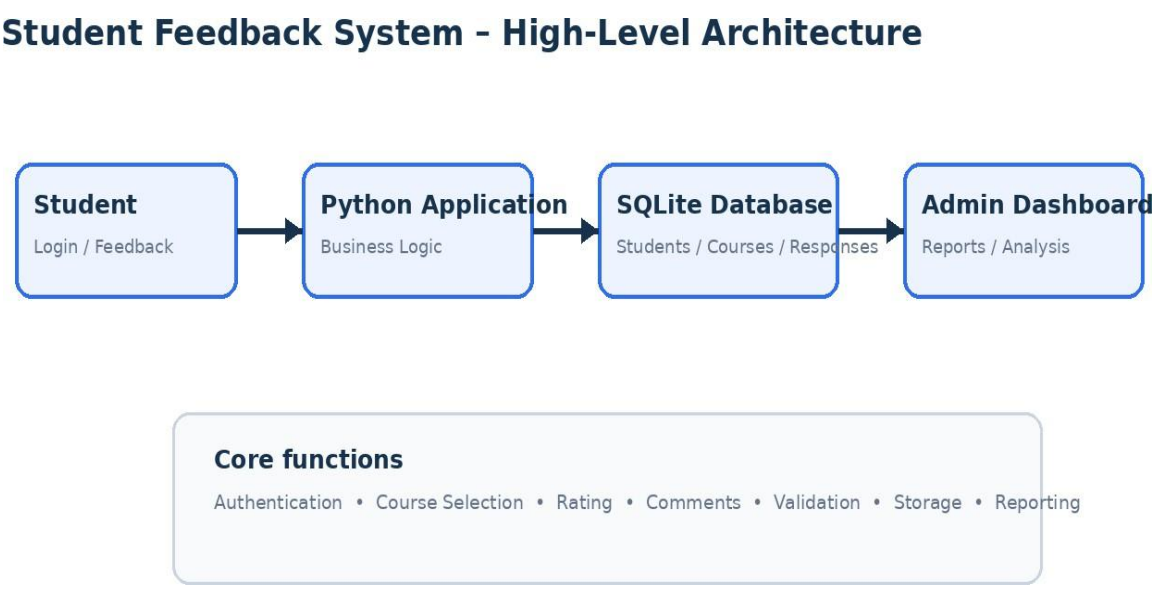


Figure 3.1: High-level architecture of the Student Feedback System

The architecture separates user interaction from business processing and data persistence. A student interacts with the application through the GUI. The Python application validates the request and performs database operations. Administrative functions use stored responses to produce summaries.

3.1.1 Feedback Submission Flow

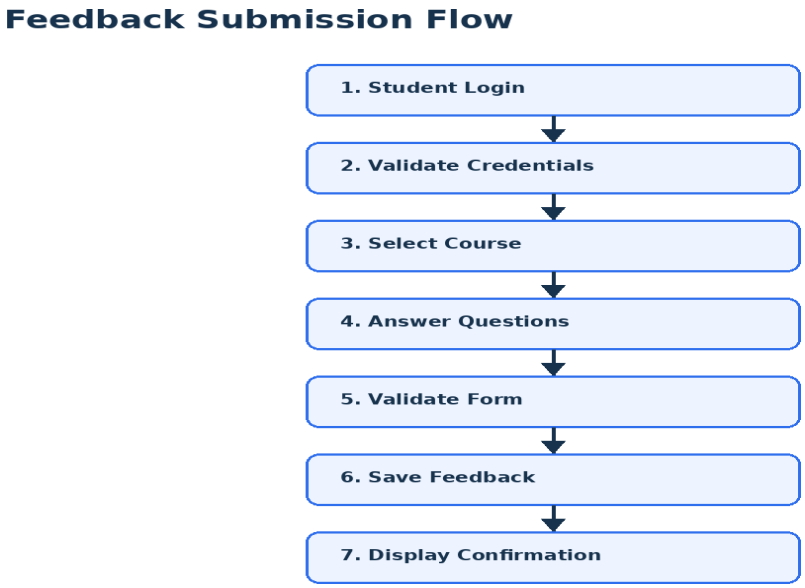


Figure 3.2: Feedback submission workflow

The workflow begins with authentication and continues through course selection, question answering, validation, database insertion, and confirmation. Validation is performed before the database transaction to reduce invalid records.

3.1.2 Database Design

Table	Important Fields	Purpose
students	student_id, name, password_hash	Stores registered student accounts
courses	course_id, course_name, faculty_name	Stores courses available for feedback
feedback	feedback_id, student_id, course_id, rating, co	Student details, submitted responses
admins	admin_id, username, password_hash	Stores administrative accounts

3.2 Modular Breakdown

Module	Responsibilities
Authentication	Login, role identification, credential validation
Student Dashboard	Navigation and feedback status
Feedback Form	Question display, rating selection, comments
Validation	Required-field checks and rating range checks
Database Layer	Create tables, insert, update, select, aggregate
Admin Dashboard	View response counts and average ratings
Reporting	Prepare summary information for review

3.3 Use Case Description

Actor	Use Case	Outcome
Student	Login	Authenticated access to student functions
Student	Submit Feedback	Response stored successfully
Student	View Status	Submitted/pending information displayed
Admin	View Feedback	Responses retrieved from database
Admin	Analyze Results	Average ratings and summaries displayed

CHAPTER 4: DEVELOPMENT AND RESULT ANALYSIS

4.1 System Implementation and Development

The implementation was organized into reusable functions so that interface code and database operations could be maintained independently. The prototype uses Tkinter for the graphical interface and sqlite3 for database access. The database is initialized when the application starts, after which the user can authenticate and submit feedback.

4.1.1 Database Initialization – Sample Code

```
import sqlite3
connection = sqlite3.connect("feedback.db") cursor =
connection.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS feedback (
id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id TEXT NOT NULL, course TEXT NOT NULL, rating
INTEGER NOT NULL,
comment TEXT,
submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) """)
connection.commit() connection.close()
```

The database initialization creates the feedback table if it does not already exist. In a production system, database credentials, migrations, access control, backup procedures, and stronger validation should be added.

4.1.2 Feedback Submission – Sample Code

```
def save_feedback(student_id, course, rating, comment): if not
  student_id or not course:
  raise ValueError("Required fields are missing")
  if rating not in range(1, 6):
  raise ValueError("Rating must be between 1 and 5")
  conn = sqlite3.connect("feedback.db") cur = conn.cursor()
  cur.execute(
  """INSERT INTO feedback
  (student_id, course, rating, comment) VALUES (?, ?, ?, ?)""",
  (student_id, course, rating, comment)
  )
  conn.commit() conn.close()
  return "Feedback submitted successfully"
```

Parameterized SQL values are used in the example instead of concatenating raw user input into an SQL statement. The application should also use secure password handling and role-based authorization in any production deployment.

4.1.3 Average Rating – Sample Code

```
  def get_average_rating(course):
  conn = sqlite3.connect("feedback.db") cur = conn.cursor()
  cur.execute(
  "SELECT AVG(rating) FROM feedback WHERE course = ?",
  (course,))
  result = cur.fetchone()[0]
  conn.close()
  return round(result or 0, 2)
```

4.1.4 Sample Output Screenshots

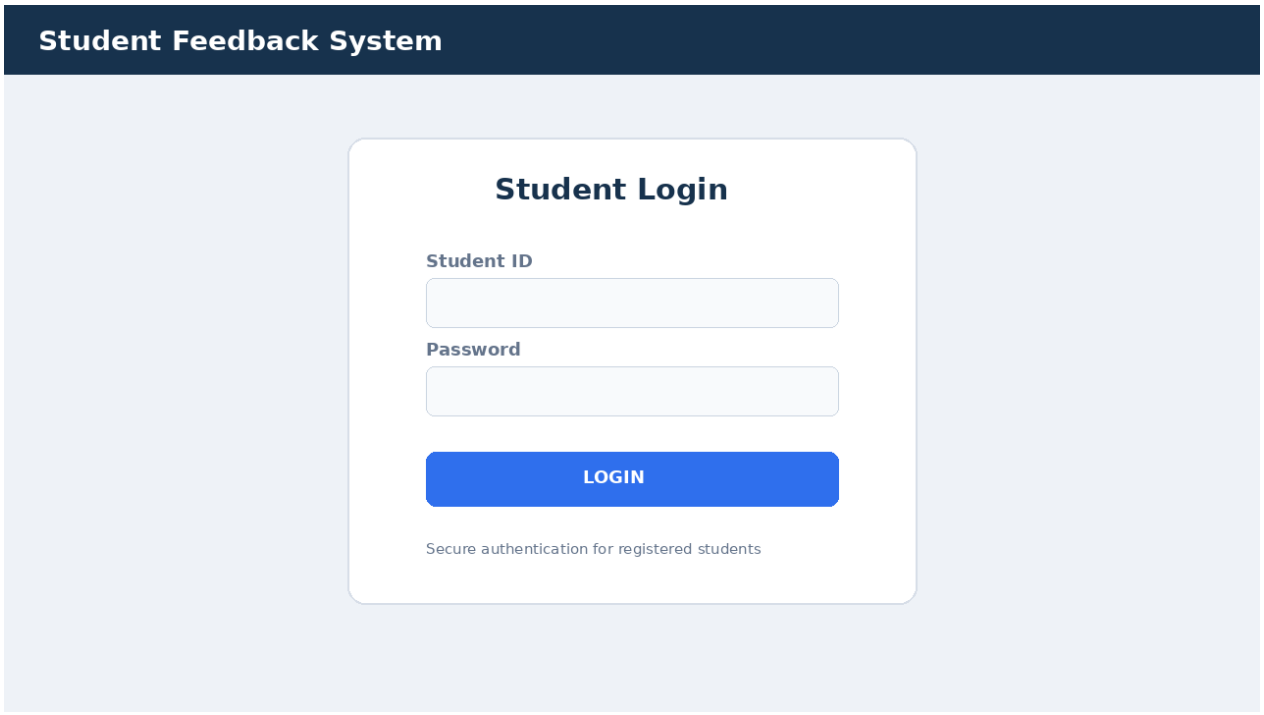


Figure 4.1: Sample Student Login screen

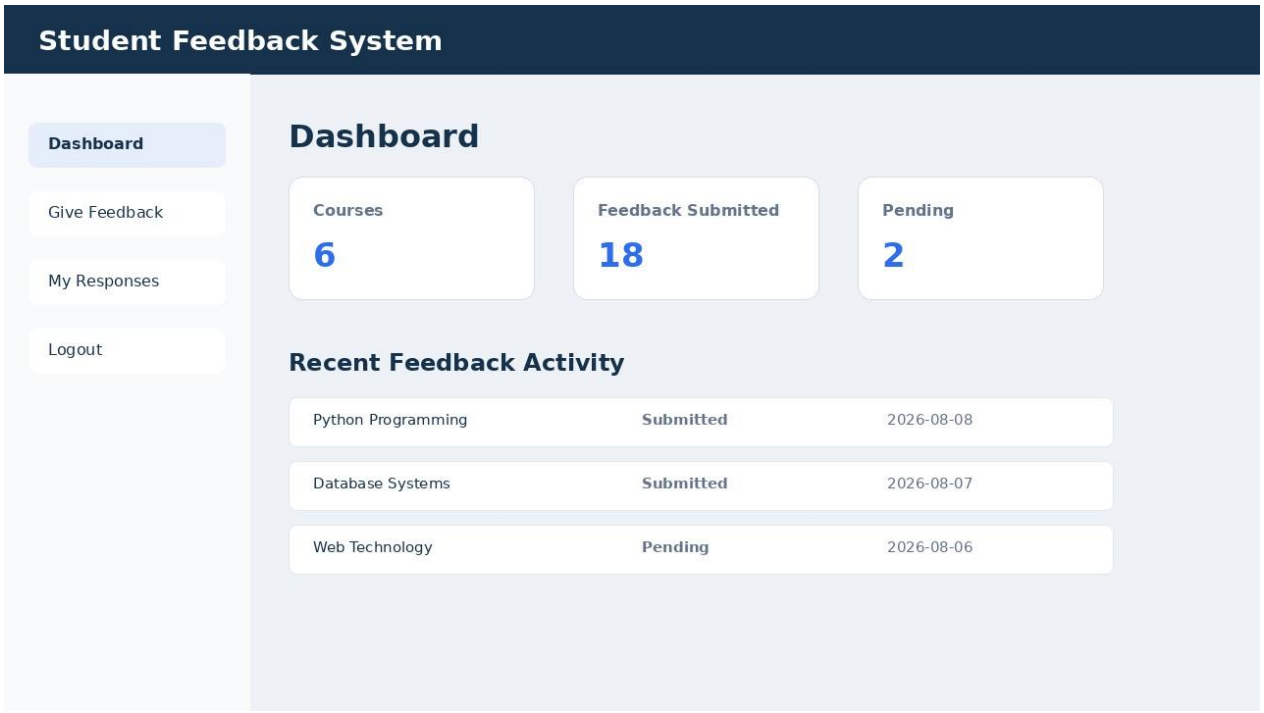


Figure 4.2: Sample Student Dashboard screen

4.1.5 Feedback and Administration Screens

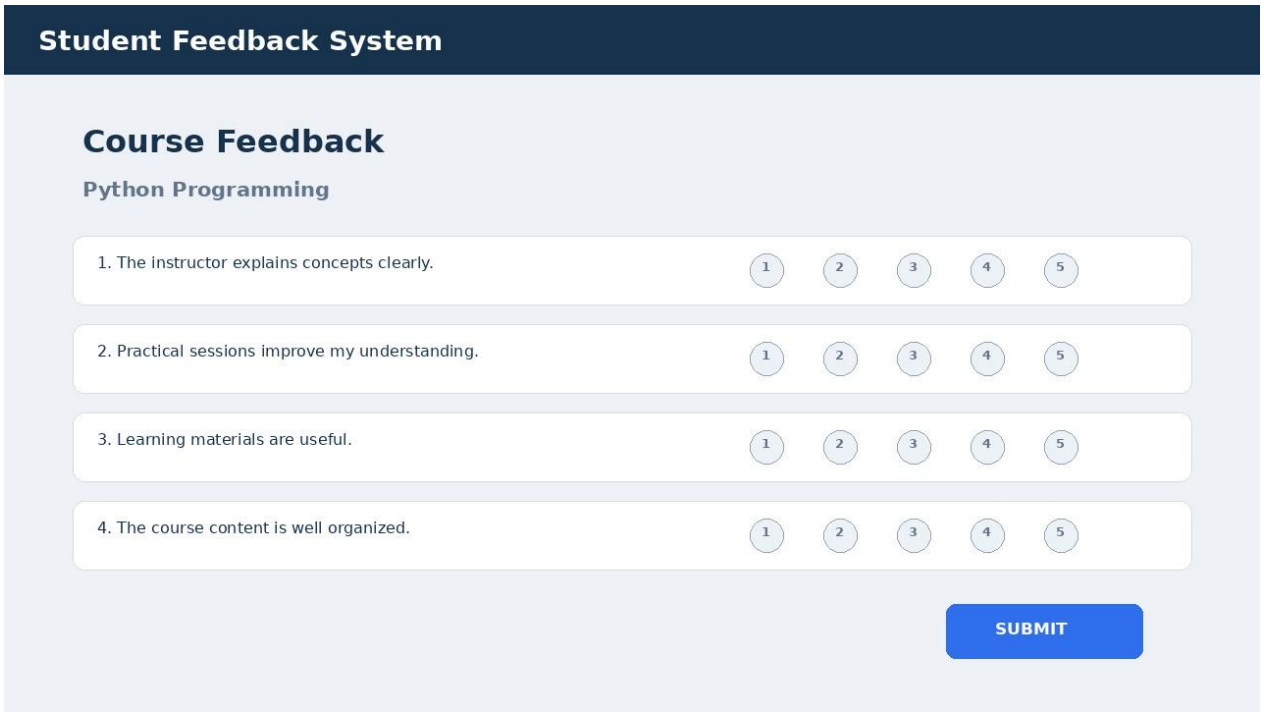


Figure 4.3: Sample Course Feedback screen

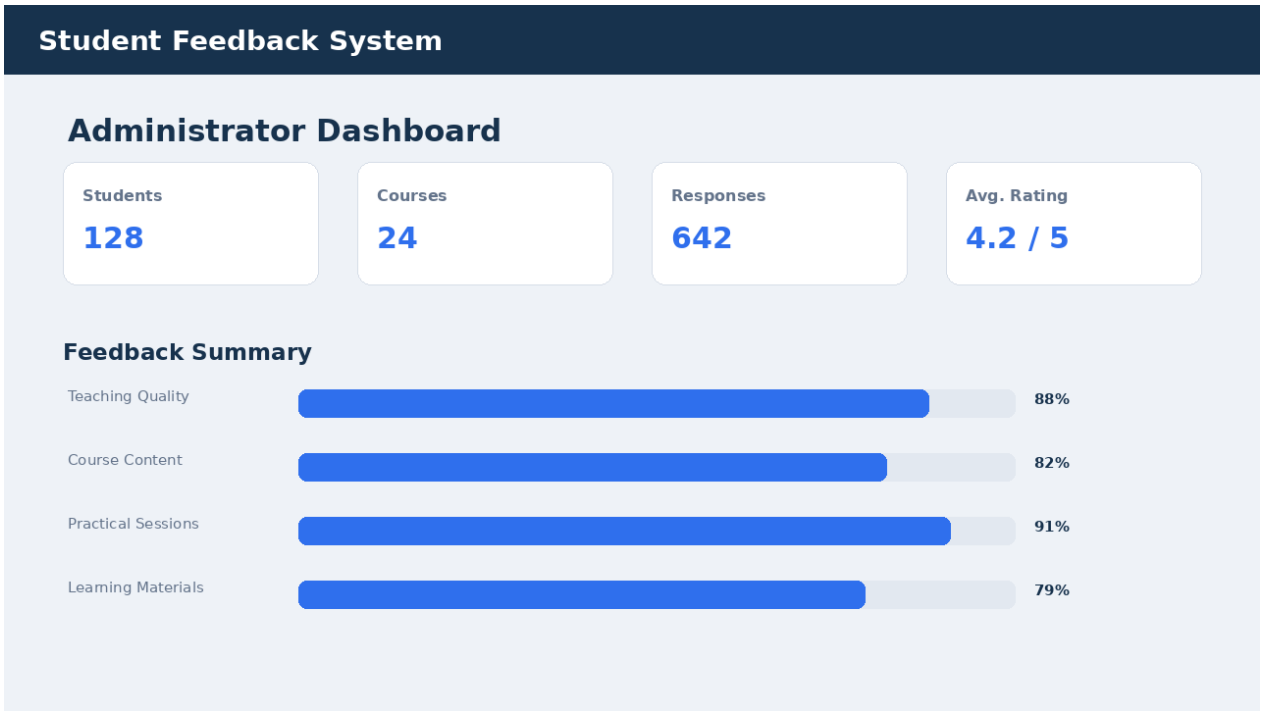


Figure 4.4: Sample Administrator Dashboard

4.2 Result Analysis and Testing Parameters

Testing was performed by checking the major functional paths of the prototype. The objective was to confirm that valid input is accepted, invalid input is rejected, and successful feedback is stored correctly.

Test ID	Test Case	Expected Result	Status
TC-01	Valid student login	Dashboard opens	Pass
TC-02	Invalid login	Error message displayed	Pass
TC-03	Course selected	Feedback form opens	Pass
TC-04	Missing rating	Submission blocked	Pass
TC-05	Rating outside 1–5	Validation error	Pass
TC-06	Valid feedback submission	Record stored	Pass
TC-07	Admin opens summary	Statistics displayed	Pass
TC-08	Average rating calculation	Correct aggregate shown	Pass

4.2.1 Sample Result Dataset

Course	Responses	Average Rating	Interpretation
Python Programming	105	4.3	Very Good
Database Systems	98	4.1	Very Good
Web Technology	110	4.0	Good

Software Engineering	92	4.2	Very Good

The values above are illustrative sample data for demonstrating report presentation and should be replaced with actual internship project results if measured from the implemented application.

CHAPTER 5: CONCLUSION

5.1 Conclusion

The Student Feedback System using Python demonstrates how a repetitive academic feedback process can be converted into a structured software application. The project integrates user authentication, feedback collection, validation, database storage, and summary analysis within a simple and maintainable architecture.

The internship helped strengthen practical skills in Python programming, GUI design, SQL-based database operations, modular development, debugging, testing, and technical documentation. The project also highlighted the importance of requirement analysis and validation before implementation.

Limitations

- The prototype is primarily intended for a small-scale environment.
- The desktop interface requires local installation and is not a multi-user web deployment.
- Advanced analytics and visualization are limited in the prototype.
- Production deployment would require stronger authentication, authorization, logging, backup, and security controls.

5.2 Future Enhancements

- Convert the application into a web system using Flask or Django.
- Use PostgreSQL/MySQL for larger institutional deployments.
- Add role-based access control for students, faculty, and administrators.
- Add charts, trend analysis, downloadable reports, and course-wise comparisons.
- Add email or in-app notifications for feedback cycles.
- Introduce automated testing and continuous integration.

REFERENCES

[1] Python Software Foundation, Python 3 Documentation – The Python Tutorial, Python Documentation, 2026.

[2] Python Software Foundation, tkinter — Python interface to Tcl/Tk, Python Standard Library Documentation, 2026.

[3] Python Software Foundation, sqlite3 — DB-API 2.0 interface for SQLite databases, Python Standard Library Documentation.

[4] Python Software Foundation, About the Python Software Foundation, 2026.

[5] Python Software Foundation, Mission of the Python Software Foundation, 2026.

[6] G. Van Rossum and the Python development community, Python language and standard library documentation, Python Software Foundation.

[7] SQLite documentation, database concepts and SQL reference materials.

[8] General software engineering references on requirements analysis, modular design, database systems, GUI design, and software testing.

ANNEXURE

Internship (June – July 2026)

1. Name of the Industry with address:
Fantasy Solution, No 16, Samnath Plazza, Third Floor, Near BG Naidu Sweets, Melapudur, Trichy-1.

2. Name of the Industrial Expert (Joint Supervisor): Mohamed Ibrahim

3. Qualification of the Industrial Expert (Joint Supervisor): MCA

4. Designation of the Industrial Expert (Joint Supervisor): Software Developer

5. Title of the Project Work:

Student Feedback System using python

6. Duration of the Internship: 1st June 2026 – 30th June 2026

7. Details of the student:

Sl. No.	Name of the student	Registration No.	Department
01	KESAVAN E	810023205087	IT

Terms and conditions

- The student is permitted to undergo a semester long internship in your organization
- The student shall be instructed to meet the Institute supervisor periodically and to attend the review committee meetings for evaluating the progress.
- The final report of the internship duly signed by the industrial expert/supervisor will be submitted to college.
- Attendance Certificate in the prescribed format signed by the competent authority of the industry will be produced
- Internship Certificate in the prescribed format signed by the competent authority of the industry will be produced

Seal of the Organization

Signature and Designation of the Officer

FANTASY SOLUTION

Fins Ur Passion...

SOFTWARE DEVELOPMENT AND TRAINING

Certificate of Accomplishment

This is certify that Mr./Miss. KESAVAN E (8100238205087)

Student of UNIVERSITY COLLEGE OF ENGINEERING BIT CAMPUS ANNA UNIVERSITY -TRICHY

underwent Inplant Training / Internship / Course on PYTHON PROGRAMMING

concern from 01/06/2026 to 30/06/2026 issued on 30/06/2026

Technical Incharge

Manager

CEO

FANTASY SOLUTION TRICHY-1

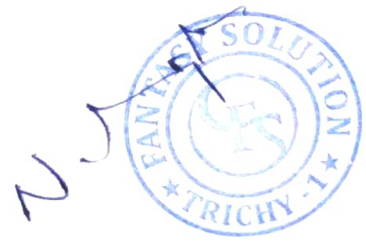
#16, Samnath Plazza, 3rd Floor, Madurai Main Road, Melapudur, Trichy-1.
0431 358 3860 fantazysolution@gmail.com www.fantasysolution.in

ATTENDANCE CERTIFICATE FOR INDUSTRIAL TRAINING / INTERNSHIP

(Attendance Certificate to be signed by the competent authority of the industry mentioning the period of Industrial Training / Internship)

The Student Mr./ Ms. **KESAVAN E** (Reg.No.) **810023205087**,

Studying **PYTHON** programme at **UNIVERSITY COLLEGE OF ENGINEERING BIT CAMPUS ANNA UNIVERSITY, TIRUCHIRAPPALLI** College in semester **7th** at Department of **INFORMATION TECHNOLOGY** has attended Industrial Training / Internship from **01/06/2026** to **30/06/2026**. It is certified that he / she has completed / not completed the Industrial Training / Internship in our **FANTASY SOLUTION** organization / institution.



Signature

Name and Designation of the Officer
Seal of the Organization